

Field Notes from a Provincial System

Errors, crashes and silent failures in a multi-platform library network – and how each one was found

Drawn from the development of NEXLIB, a library management system for Government Degree Colleges under the Higher Education Department, Khyber Pakhtunkhwa.

Four applications · ~57,700 lines · Android, Windows, and two web portals.

Preface: why these notes exist

Most books about debugging are written backwards. The author knows the answer before the first sentence, so the bug arrives already labelled, the investigation runs in a straight line, and the reader never experiences the thing that actually makes debugging hard: not knowing, for hours, which of forty possibilities is true.

This book is assembled from a real project's wreckage. Every failure in it happened, in order, to a working system with real users and real library data. The diagnoses are the ones that were actually reached – including the ones that were wrong first, and had to be abandoned.

NEXLIB is a library management system for the government degree colleges of Khyber Pakhtunkhwa. It is four separate applications – an Android app, a Windows desktop program, a college web portal and a directorate portal – sharing one multi-tenant cloud database, and designed to keep working in colleges where the internet does not. That shape generates a particular and instructive class of bug: faults that appear on one platform and not another, faults that are invisible until a second device disagrees, and faults that never crash anything at all.

Three convictions run through what follows.

A crash is the best kind of failure. It is loud, it names a file and a line, and it stops before doing more damage. The failures that cost this project the most time never crashed: a delete that quietly did nothing, a status badge that reported the wrong problem, a wipe that reported success while leaving most of the data behind. Chapter after chapter here is about failures that looked like nothing.

The error message is usually not the error. `Connection to the Kotlin daemon has been unexpectedly lost` is an out-of-memory condition. `INVALID_LOGIN_CREDENTIALS` frequently means the account does not exist. A library book reappearing after deletion is a query that excluded the wrong rows. Learning to read past the message to the mechanism is most of the skill.

Architecture decides which bugs are possible. The desktop app talks to the database through an administrative credential that bypasses every security rule. That one decision made an entire category of fault invisible on the platform the developer used daily, and guaranteed it would appear only on the platforms he could not easily test. No amount of care compensates for that; only changing the architecture does.

The chapters are grouped so they can be read out of order. Each case study follows the same shape: the symptom as it was first seen, the false trails, the mechanism, the fix, and what it should have taught us earlier.

Contents

Chapters marked *written* are complete in this edition. The remainder are outlined and drafted in sequence.

PART I · THE SYSTEM AND ITS SHAPE

1 Four applications, one database

OUTLINED

2	What multi-tenancy actually requires	OUTLINED
3	Offline-first: the inversion that changes everything	OUTLINED
PART II · CRASHES		
4	What a crash actually is	WRITTEN
5	Two crashes, dissected	WRITTEN
6	The crash that was not a crash	WRITTEN
7	Reading a traceback you did not write	OUTLINED
8	Crashes that only happen after packaging	OUTLINED
PART III · FAILURES THAT NEVER CRASH		
9	The cost of catching everything	WRITTEN
10	The delete that came back	WRITTEN
11	The status badge that blamed the network	OUTLINED
12	Phantoms: deleting what is already gone	OUTLINED
13	Last-write-wins and the clock you do not control	OUTLINED
PART IV · SECURITY FAILURES		
14	One rule, every college's data	OUTLINED
15	The button that made everyone an administrator	OUTLINED
16	Shipping a key that unlocks everything	OUTLINED
17	Testing rules instead of trusting them	OUTLINED
PART V · SCALE, COST AND MEMORY		
18	Read amplification: the 155-fold bill	OUTLINED
19	Why the catalogue size was bounded by RAM	OUTLINED
20	Measuring instead of projecting	OUTLINED
PART VI · BUILD AND DELIVERY		
21	Builds that fail for reasons unrelated to code	OUTLINED
22	Wiping a system so it stays wiped	OUTLINED
23	The delivery checklist	OUTLINED
PART VII · PRACTICE		
24	Building a diagnostic tool	OUTLINED

25	Instrumenting a system you cannot visit	OUTLINED
26	What we would do differently	OUTLINED

PART II

Crashes

The failures that announce themselves – and are therefore the easiest kind to have.

CHAPTER 4

What a crash actually is

Before dissecting particular crashes it is worth being precise about what one is, because the word covers at least four different events that need different responses.

In a Python desktop application, a crash is usually an *unhandled exception*: some function raised, no enclosing block caught it, and the interpreter unwound the stack to the top and stopped. In a Kotlin Android application it is much the same, with the operating system killing the process at the end. But "the app crashed" is also said about a process killed by the operating system for using too much memory, about a window that stopped repainting, and about a program that exited cleanly for reasons the user did not understand. These have almost nothing in common except the user's experience of them.

The distinction matters because only the first leaves evidence.

The global handler, and why it is not optional

A GUI application without a top-level exception handler simply vanishes. The user sees the window disappear; there is no message, no log, nothing to report. The desktop half of NEXLIB installs one at startup:

```
def _global_exception_handler(exc_type, exc_value, exc_tb):
    """Show a dialog for unhandled exceptions instead of silently crashing."""
    msg = "".join(traceback.format_exception(exc_type, exc_value, exc_tb))
    logger.critical(f"Unhandled exception: {msg}")
    try:
        dlg = QMessageBox()
        dlg.setWindowTitle("Unexpected Error")
        dlg.setText("An unexpected error occurred. The application may be unstable.")
        dlg.setDetailedText(msg)
        dlg.setIcon(QMessageBox.Icon.Critical)
        dlg.exec()
    except Exception:
        pass

sys.excepthook = _global_exception_handler
```

gdc_desktop/main.py

Three things in that handler are worth noting, because each was learned rather than designed.

It **logs before it displays**. Showing a dialog requires the Qt event loop to be in a sane state, and the thing that just crashed may have left it otherwise. Writing to the log first means the evidence survives even when the dialog cannot be drawn.

It puts the traceback in `setDetailedText` rather than the main message. A librarian sees one plain sentence; the person diagnosing it clicks "Show Details" and gets the whole stack. A dialog that dumps a traceback at a non-technical user teaches them to dismiss dialogs without reading.

It **wraps its own dialog in a try/except**. A crash handler that crashes produces the original silent disappearance plus a confusing second failure.

The handler that does not catch everything

Here is the limitation that cost this project real time. `sys.excepthook` is called for unhandled exceptions *on the main thread*. An exception raised inside a `QThread` worker never reaches it.

NEXLIB has twenty-two files that use `QThread`, and background workers are precisely where the risky work lives: network calls, Firestore syncs, report generation, bulk imports. The application's most failure-prone code was the part the crash handler could not see. A sync worker could raise, die, and leave the user with a library that simply stopped synchronising – no dialog, no log entry, nothing.

The fix is a second handler, introduced in Python 3.8:

```
threading.excepthook = _thread_exception_handler
```

Any platform with background threads has this same hole. Android needs

`Thread.setDefaultUncaughtExceptionHandler`; JavaScript needs both `window.onerror` and a separate `unhandledrejection` listener, because a rejected promise is not an error event and the two are reported through different channels.

LESSON

Count your entry points for failure, not just your handlers. Every thread, every event loop, every promise chain is a place an exception can escape, and each needs its own catch of last resort.

What a crash log is for

A log that nobody reads is not instrumentation; it is a file. The desktop app wrote crashes faithfully to `gdc_library.log` for months, and in that time nobody opened it once – not from negligence, but because a log is an undifferentiated stream and the interesting part is buried.

What made the log useful was grouping. One bug that fires forty times produces forty tracebacks, which reads as catastrophe; grouped by fault it reads as one problem with a count, which is what it is:

```
CRASHES – 2 total, 1 distinct
```

```
2 x ValueError: Invalid image mode
```

```
ERRORS – 2 total, 1 distinct
```

```
2 x Push failed: DEADLINE_EXCEEDED
```

```
first 2026-09-20 09:30:00, last 2026-09-20 11:30:00
```

Two numbers do most of the work here. *Distinct* tells you how many problems you have. *Total* tells you how much they are hurting. A fault with a count of two and a fault with a count of four hundred deserve very different attention, and the raw log makes that distinction almost impossible to see.

There is a subtlety in the grouping worth recording, because getting it wrong made the first version of that report useless. The global handler writes a line beginning `Unhandled exception:` followed by the traceback. A naive error tally counts that line too — so every crash appeared twice: once under its real fault, and once under the generic heading that names nothing. The tally has to skip the line that opens a traceback, because it is already counted by the fault it introduces.

The chapters that follow examine specific crashes. Keep two questions in mind for each: *what did the user actually see?* and *what evidence survived?* In nearly every case the gap between those two answers is where the time went.

CHAPTER 5

Two crashes, dissected

These two arrived on the same afternoon, in the same application, and looked identical to the user: a dialog saying "Unexpected Error". Their causes had nothing in common, and the difference between them is instructive.

Case one: a variable that was never defined

Symptom. A librarian clicks "Generate Full Analytics PDF". The dialog appears. No report is produced.

Evidence. The detailed text showed:

```
NameError: name 'gold' is not defined
  File "services/report_service.py", line 333, in generate_full_analytics_report
```

Mechanism. Line 333 used a colour variable named `gold` to draw a horizontal rule. Two other report functions in the same file defined `gold` as a local at the top of their bodies. This one did not. The function had been written by copying a neighbour and editing the middle, and the definition lived above the copied region.

This is as simple as a bug gets, and it survived into a shipped application for a reason worth naming: **Python does not check that a name exists until the line runs**. Any function whose unhappy path is never exercised can carry a `NameError` indefinitely. The report generated correctly for months because line 333 sat inside a branch that only executed when the analytics section had data to draw a rule under.

Fix. Define the local, as its neighbours do.

The interesting question is not how to fix it but how it was allowed to exist. A linter would have caught this in under a second – `pyflakes` reports undefined names without running anything. The project had no linter and no continuous integration. One tool, configured once, would have found this class of fault across all 20,000 lines of Python.

LESSON

In a dynamically typed language, unexecuted code is unverified code. Static analysis is not a luxury; it is the only thing standing between you and a `NameError` in a branch nobody has taken yet.

Case two: an image in the wrong mode

Symptom. Opening the Digital ID Cards screen produced the same "Unexpected Error" dialog. The screen never appeared at all.

Evidence. The traceback ended inside Qt's image conversion, called from a function that turns a member's ID into a QR code.

The false trail. The obvious first hypothesis was a bad member record – a null field, an unprintable character in a name. That hypothesis is attractive because it is the kind of bug one expects, and it wasted time. The crash occurred for every member, including ones whose records were plainly clean.

Mechanism. The `qrcode` library returns a PIL image in mode `"1"` – one bit per pixel, the natural representation for something that is only ever black or white. Qt's `ImageQt` bridge accepts several PIL modes, but support for 1-bit images varies across combinations of Pillow and Qt versions. On this machine's combination, it did not.

So the code was correct, the data was correct, and the crash was real. What failed was an assumption about a dependency's behaviour that had held on the developer's previous environment and stopped holding on this one.

Fix. Convert to a mode every version supports before handing the image to Qt:

```
# qrcode returns a 1-bit ("1" mode) image, which PIL's ImageQt does not
# support on all Pillow/Qt combinations – convert to RGB first.
img = qr.make_image(fill_color="black", back_color="white").convert("RGB")
```

`gdc_desktop/ui/screens/digital_id_screen.py`

A second change mattered as much as the conversion: the call to `generate_qr` was wrapped so that one member whose QR cannot be built does not prevent the whole screen from opening. Before, a single bad record took down the entire Digital ID feature. After, that member's card is blank and the other fifteen hundred print.

LESSON

Decide the blast radius deliberately. When one item in a list fails, the question "should this kill the list?" has a right answer, and it is usually no. Failing the whole screen is a decision, even when it is made by omission.

What the two cases share

Nothing, in their mechanisms. But both were in code paths that no automated test touched, in an application with four unit tests across 57,700 lines. Both were found by a user clicking a button.

That is the real finding, and it recurs throughout this book: the project's defect rate was not unusual. Its *detection* rate was. Faults were discovered in the order that users happened to encounter them, which is the most expensive possible order.

CHAPTER 6

The crash that was not a crash

Some failures lie about their own nature. This one presented as a crashed process, was reported as a lost connection, and was in fact a machine running out of memory.

Symptom. Building the Android application failed:

```
Daemon compilation failed: Connection to the Kotlin daemon has been
unexpectedly lost. This might be caused by the daemon being killed by
another process or the operating system, or by JVM crash.
```

```
org.jetbrains.kotlin.gradle.tasks.DaemonCrashedException
```

What the message suggests. Read plainly, it proposes three causes: another process killed the daemon, the operating system killed it, or the JVM crashed. All three sound like environmental misfortune – antivirus interference, a flaky machine, a corrupted installation. None sounds like "configuration".

This framing is actively misleading, and it is worth pausing on why. The Gradle process is not the process that died. It observed a socket close and reported what it could observe. The dying process had no opportunity to explain itself, because a JVM terminated by the operating system's memory manager does not get to write a message. So the only account available comes from a witness who saw nothing but the silence.

Mechanism. The project configured the Kotlin compiler daemon with a one-gigabyte heap:

```
org.gradle.jvmargs=-Xmx1536m -XX:MaxMetaspaceSize=512m
kotlin.daemon.jvmargs=-Xmx1024m
```

One gigabyte is generous for a small module and inadequate for this one. The shared module compiles Jetpack Compose, Hilt with its annotation processors, SQLDelight's generated sources and Apache POI in a single unit. The compiler exhausted its heap, the JVM died, the socket closed, and Gradle reported a lost connection.

There is a second resource that fails the same way and is easy to overlook. Metaspace holds class metadata, and annotation processors load an enormous number of classes. A daemon can exhaust metaspace while heap sits comfortably under its limit, producing the identical symptom. The original configuration set no metaspace limit for the daemon at all.

Fix.

```
org.gradle.jvmargs=-Xmx2560m -XX:MaxMetaspaceSize=768m -XX:+UseParallelGC
kotlin.daemon.jvmargs=-Xmx2048m -XX:MaxMetaspaceSize=512m -XX:+UseParallelGC
```

And an operational detail that wastes an afternoon when missed: **a running daemon keeps the arguments it started with**. Editing the configuration changes nothing until the existing daemon is stopped. `gradlew --stop` is not housekeeping; it is the step that makes the fix take effect.

When more memory is not available

Raising the limits got the build further – to a genuine compilation error rather than a dead socket – but it failed again on the same machine, because that machine was also running an IDE holding two gigabytes of its own. Two JVMs wanting 2.5 GB and 2 GB on a machine that did not have them simply moved the problem.

The second fix removes the second process entirely:

```
kotlin.compiler.execution.strategy=in-process
```

The compiler now runs inside the Gradle daemon. Cold builds are slower, because the compiler is no longer kept warm between runs. In exchange, there is no separate process to lose a connection to – one process cannot lose a connection to itself. For a build that otherwise cannot finish, that is an easy trade.

LESSON

When an error message describes a relationship failing – a lost connection, a closed pipe, a timeout – ask what happened to the thing at the other end. The reporter is rarely the casualty, and the casualty rarely gets to file a report.

A note on believing build errors

The same build later failed with `Compilation error. See log for more details` and no `e:` line naming a file – which is what Kotlin emits when the daemon dies mid-compilation. It looked like a code error and was another instance of the same memory problem wearing a different costume.

The discriminator is simple and worth memorising: **a real Kotlin compilation error always names a file and a line**. A "compilation error" with nothing to point at is not a compilation error.

PART III

Failures That Never Crash

The expensive ones. No dialog, no traceback, no log entry – just a system quietly doing the wrong thing.

CHAPTER 9

The cost of catching everything

A count, to begin with. Across the four applications there are 357 broad exception handlers: 180 in Python, 149 in Kotlin, 28 in TypeScript. That is roughly one for every 160 lines of code.

Each one was written for a good reason. A network call might fail; a malformed record might not parse; a file might be missing. Catching the exception keeps the application running instead of dropping the librarian back to a blank screen. The applications feel stable, and that stability is real.

But an exception handler is a decision about what should happen next, and the overwhelming majority of these made the same decision:

```
try:
    do_the_thing()
except Exception:
    pass
```

Which says: *if this fails, proceed as though it succeeded.*

That is occasionally correct. Failing to write an optional cache entry should not stop a sale. But applied broadly it converts a system that fails loudly into one that fails silently, and silent failure in a data system is not a smaller problem than a crash – it is a larger one, because it is discovered later and by someone with less information.

The shape of the damage

Consider a sync worker that pushes pending records to the cloud. Wrapped in a broad handler, a failure at any point produces: no crash, no dialog, no log entry the user would see, and a status badge that continues to read *Synced* because the code that sets it was never reached by the failure path.

The librarian catalogues books for three weeks. Every record is saved locally and every record is correct. None of them reach the network. The failure surfaces when a second device shows a different catalogue, or when the directorate's totals do not match, or – worst – when the machine dies and the records that were supposedly in the cloud turn out not to be.

Three weeks of invisible divergence is a far worse outcome than an error dialog on the first day.

What to do instead

The instruction "never catch broad exceptions" is not useful advice for a real codebase; there are places where continuing is genuinely right. Three rules proved more workable.

Catch narrowly where you can name the failure. `except ConnectionError` states a belief about what can go wrong and lets everything else through. `except Exception` states no belief at all.

Never swallow silently. Even when continuing is correct, a logged line costs nothing and is the difference between a diagnosable system and an opaque one. Most of the 357 handlers would have been acceptable with a single `logger.warning` inside them.

Distinguish "failed" from "not attempted." A sync that did not run and a sync that ran and failed should never present identically. This one distinction, made explicit in the status model, resolved a fault that had consumed days – the subject of Chapter 11.

The compensating control

Retrofitting 357 handlers is not realistic. What is realistic is an external check that does not depend on any of them being correct: compare local record counts against the cloud, per collection, and report divergence.

```
[ ok ] In sync: books      4,812 both sides
[ FAIL ] Divergence: members local 1,504 vs cloud 1,171 (+333)
        - records are not reaching the network
```

This asks the only question that matters and does not care why the answer is wrong. It catches a failure in any of the 357 handlers, plus failures in code that has no handler at all, plus failures nobody anticipated.

LESSON

When you cannot trust a system to report its own failures, measure its outcomes from outside. An independent check on the result is worth more than any number of internal assertions about the process.

CHAPTER 10

The delete that came back

A librarian deletes a book on the Android app. It disappears. The next day it is back. This chapter is about why, and about a general principle of distributed systems that the bug illustrates exactly.

Symptom. Deletions performed on Android did not persist. Deletions performed on the desktop did. The book reappeared after a delay – usually overnight, sometimes within minutes – which made it look intermittent.

The false trail. Intermittency suggests a race condition, and a good deal of time went into examining sync timing. The behaviour was in fact perfectly deterministic; the *delay* was variable because it depended on when the next pull happened, but the outcome never varied at all.

Mechanism

The Android delete executed a hard SQL delete:

```
DELETE FROM books WHERE id = :id;
```

The row was gone from the device. Now consider how the push engine decides what to upload:

```
getUnsyncedBooks:  
SELECT * FROM books WHERE syncStatus = 'pending' AND deleted = 0;
```

It uploads rows. A deleted row is not a row. There was nothing left to tell the cloud about, so the cloud never learned of the deletion – and on the next pull, helpfully returned the book that the device appeared to be missing.

The system was working exactly as designed. The design was wrong.

The principle

In a system that synchronises by exchanging records, **a deletion is a fact that must itself be transmitted.** Absence cannot be transmitted; only presence can. If the only record of a deletion is the absence of a row, that deletion exists on one device and nowhere else, and any device that still has the row is correct to reinstate it.

The standard answer is a tombstone: instead of removing the row, mark it dead and let the marker replicate like any other change.

The fix, and the three things it required

Switching to a soft delete was the obvious part. Two further changes were needed, and each was its own small bug.

The tombstone must be eligible for pushing. The query above excluded `deleted = 0` – which was written to avoid uploading junk, and which guaranteed a tombstone could never be uploaded. It had to become:

```
-- Deleted rows MUST be included here: a soft delete is pushed to Firestore
-- as a tombstone (deleted = 1), and excluding them meant a delete never left
-- the device and the book reappeared on the next pull.
getUnsyncedBooks:
SELECT * FROM books WHERE syncStatus = 'pending';
```

The tombstone must be queued. The soft-delete statement marked the row dead but left `syncStatus` as it was. A row already marked `synced` stayed invisible to the push engine forever:

```
softDelete:
UPDATE books SET deleted = 1, syncStatus = 'pending', lastUpdated = :lastUpdated
WHERE id = :id;
```

Everything that reads must now exclude the dead. This is the cost of tombstones: every list, count, search and lookup has to filter them, and each place you forget is a deleted record reappearing in one particular view. Eleven queries needed the filter – `getAllBooks`, `getAvailableBooks`, `searchBooks`, `getBooksByCategory`, `getCategories`, `getTotalCount`, `getAvailableCount`, and the member equivalents.

One of them mattered more than the rest:

```
-- deleted = 0 matters here: without it a removed member could still sign in
-- to the OPAC portal with their old ID and PIN.
loginOpacStudent:
SELECT * FROM members WHERE memberId = :memberId AND pin = :pin AND deleted = 0
LIMIT 1;
```

A removed member could still authenticate. The deletion bug had quietly become an access-control bug – and would not have been found by testing deletion, because nobody thinks to test whether a deleted user can still log in.

Where tombstones are deliberately not applied

Two categories of query were left unfiltered, on purpose, and the reasoning deserves recording because "filter everywhere" is the wrong instinct.

The `*Static` variants used for backups keep full fidelity: a backup that silently drops deleted records is not a backup of the database. And lookups by id or sync id stay unfiltered because the sync engine itself needs to find tombstones in order to process them – filtering there would break the very mechanism the tombstones

exist for.

LESSON

Deleting data in a synchronised system is not the removal of a row; it is the creation of a record that says the row is gone. Systems that treat deletion as absence will always resurrect what they delete.

A closing observation

This bug existed only on Android. The desktop app had implemented soft deletes from the start. That asymmetry is characteristic of multi-platform systems: the same feature, implemented independently three times, will be correct in some implementations and wrong in others, and the ones that are wrong will be the ones the developer uses least.

Which is the subject of Part IV, where the same asymmetry produces something considerably worse than a reappearing library book.

END OF THIS EDITION

Chapters 1–3, 7–8, 11–26 in preparation

Each follows the same structure: symptom, false trails, mechanism, fix, lesson – drawn from the project's own commit history, where every case in this book is recorded with the reasoning that produced it.